

ECE 4502/6502 & CS 6501: Machine Learning on Graphs

Lecture 13. Graph Neural Networks II

Instructor: Jundong Li
Spring 2025, University of Virginia

Recap

- Deep learning for graphs
 - Desired characteristics
 - Permutation invariant and equivariant
 - Naïve approach: concatenate adjacency matrix and features
 - Sensitive to permutation and input graphs
- Graph Convolutional Networks
 - Neighborhood aggregation + neural network
 - The math behind the model

$$\mathbf{h}_v^0 = \mathbf{x}_v$$

$$\mathbf{h}_v^k = \sigma \left(\boxed{\mathbf{W}_k} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \boxed{\mathbf{B}_k} \mathbf{h}_v^{k-1} \right), \quad \forall k \in \{1, \dots, K\}$$

$$\mathbf{z}_v = \mathbf{h}_v^K$$

Matrix Formulation

- Many aggregation can be performed efficiently by (sparse) matrix operations

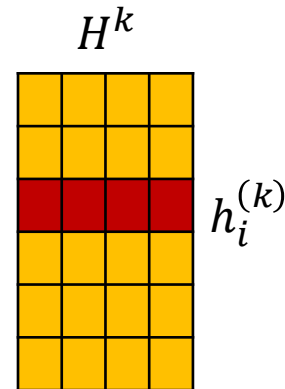
- Let $H^k = [h_1^{(k)}, \dots, h_{|V|}^{(k)}]^T$

- $\sum_{u \in N_v} h_u^{(k)} = A_{v,:} H^k$

- Let D be the diagonal degree matrix

- $D_{v,v} = |N_v|$

- D^{-1} is also diagonal with $D_{v,v}^{-1} = 1/|N_v|$



$$\sum_{u \in N_v} \frac{h_u^{(k)}}{|N_v|} \longrightarrow H^{(k+1)} = D^{-1} A H^{(k)}$$

Matrix Formulation

- Update function in matrix form
 - $H^k = \sigma(\tilde{A}H^{k-1}W_{k-1}^T + H^{k-1}B_{k-1}^T)$ where $\tilde{A} = D^{-1}A$

Neighborhood aggregationSelf transformation
- In practice, this implies that efficient sparse matrix multiplication can be used ($\tilde{A}$ is sparse)
- **Note:** not all GNNs can be expressed in a simple matrix form, when aggregation function is complex

How to Train a GNN

- Unsupervised setting:
 - No node label available
 - Use the graph structure as the supervision

- Supervised setting:

- minimize loss $\mathcal{L}$:

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f_{\Theta}(\mathbf{x}))$$

- $\mathbf{y}$: node label
 - $\mathcal{L}$ could be L2 if $\mathbf{y}$ is real number, or cross entropy if $\mathbf{y}$ is categorical

Training Unsupervised Model

- One possible idea: “Similar” nodes have similar embeddings

$$\min_{\Theta} \mathcal{L} = \sum_{z_u, z_v} CE(y_{u,v}, DEC(z_u, z_v))$$

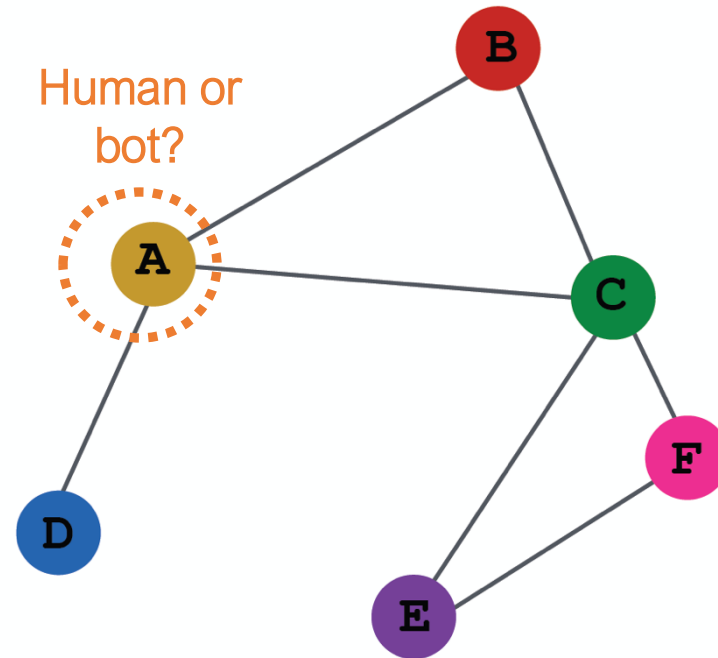
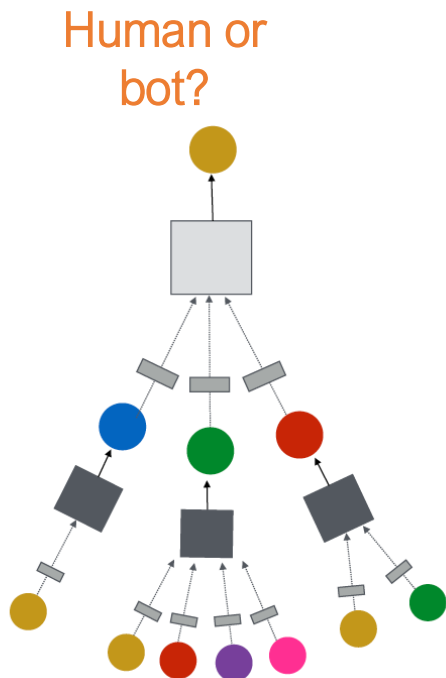
- where $y_{u,v} = 1$ when node u and v are similar
- $z_u = f_{\Theta}(u)$ and $DEC(\cdot, \cdot)$ is the dot product
- CE is the cross entropy loss:

$$CE(y, f(x)) = - \sum_{i=1}^C y_i \log f_{\Theta}(x)_i$$

- y_i and $f_{\Theta}(x)_i$ are the actual and predicted values of the i -th class.
- Node similarity can be anything from shallow embedding, e.g., a loss based on:
 - Random walks (node2vec, DeepWalk, struc2vec)
 - Matrix factorization

Training Supervised Model

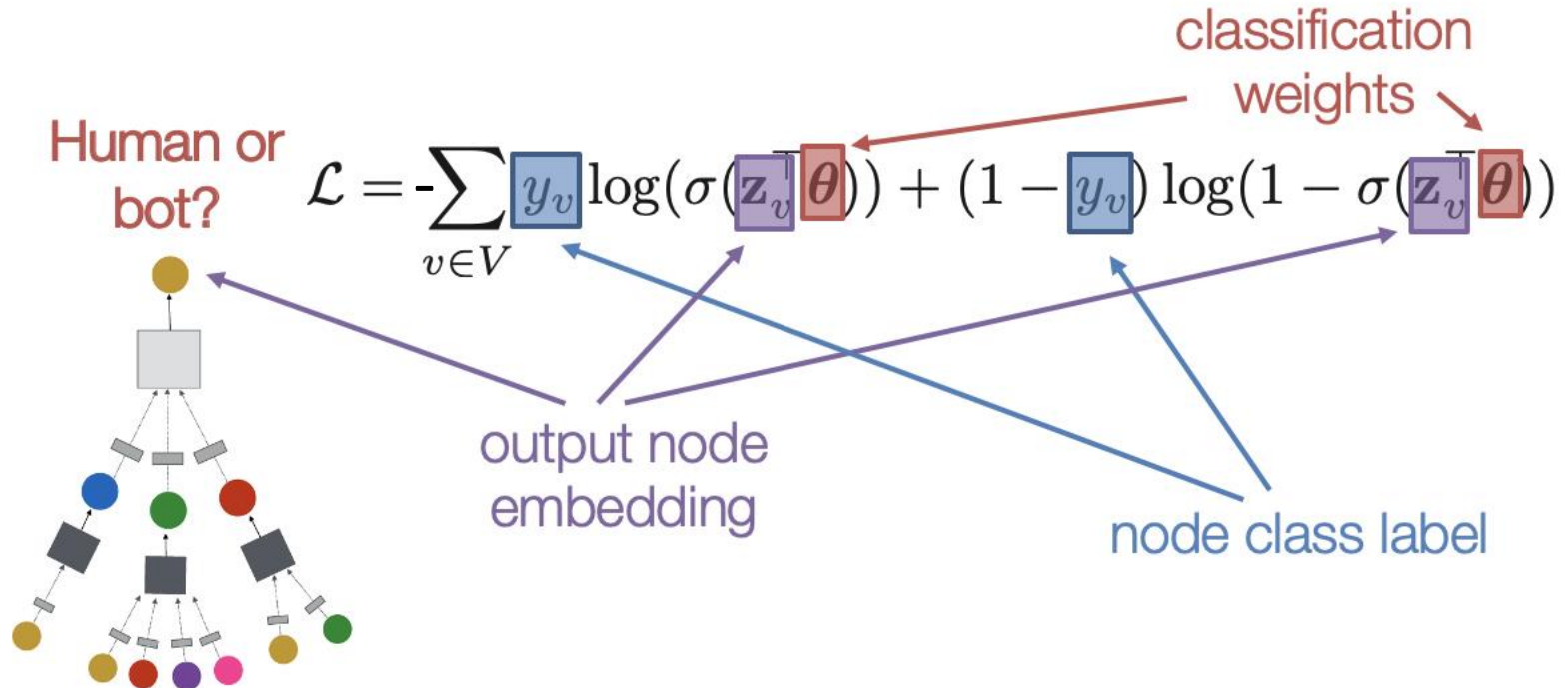
- Directly train the model for a supervised task (e.g., node classification):



e.g., an online social network
INPUT GRAPH

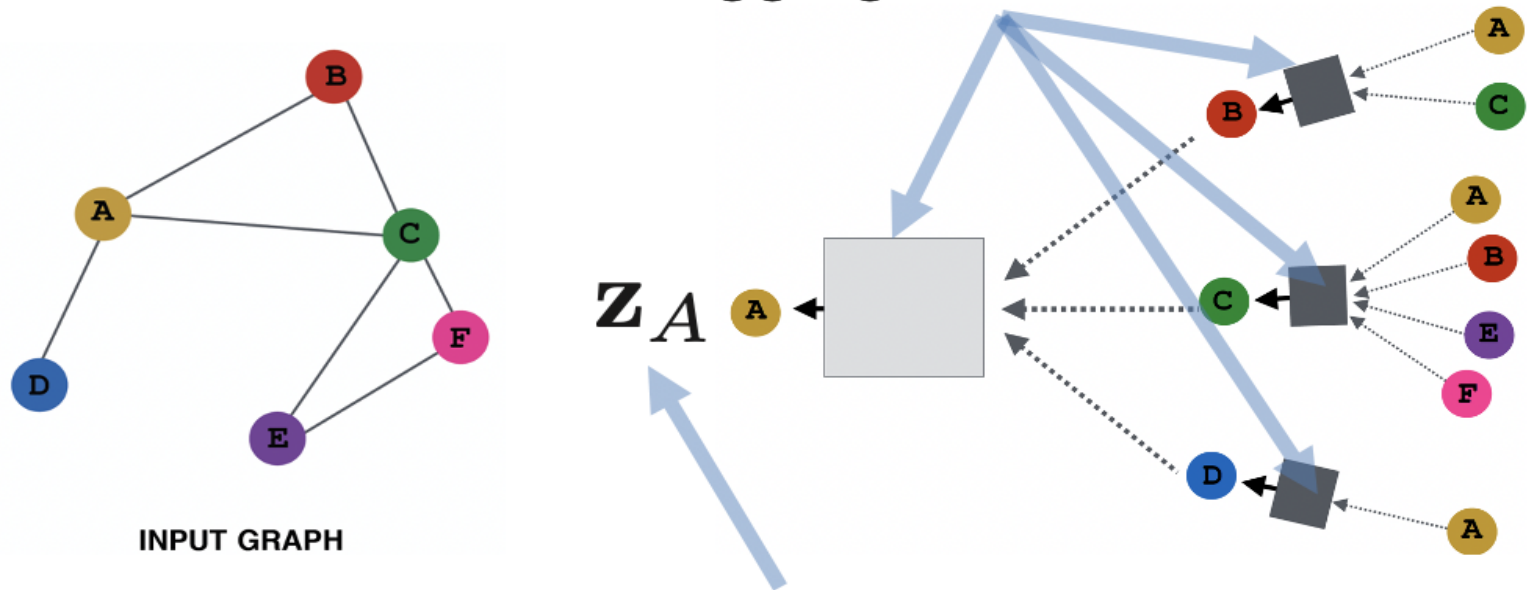
Training Supervised Model

- Use cross entropy loss



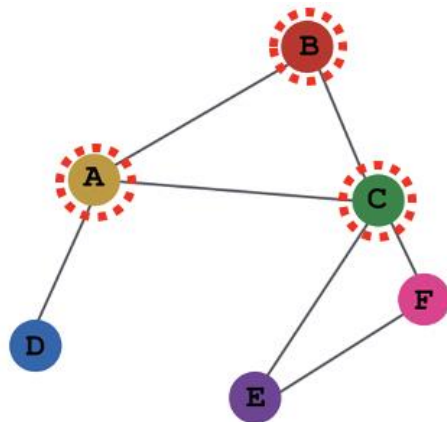
Overview of Model Design

1) Define a neighborhood aggregation function.



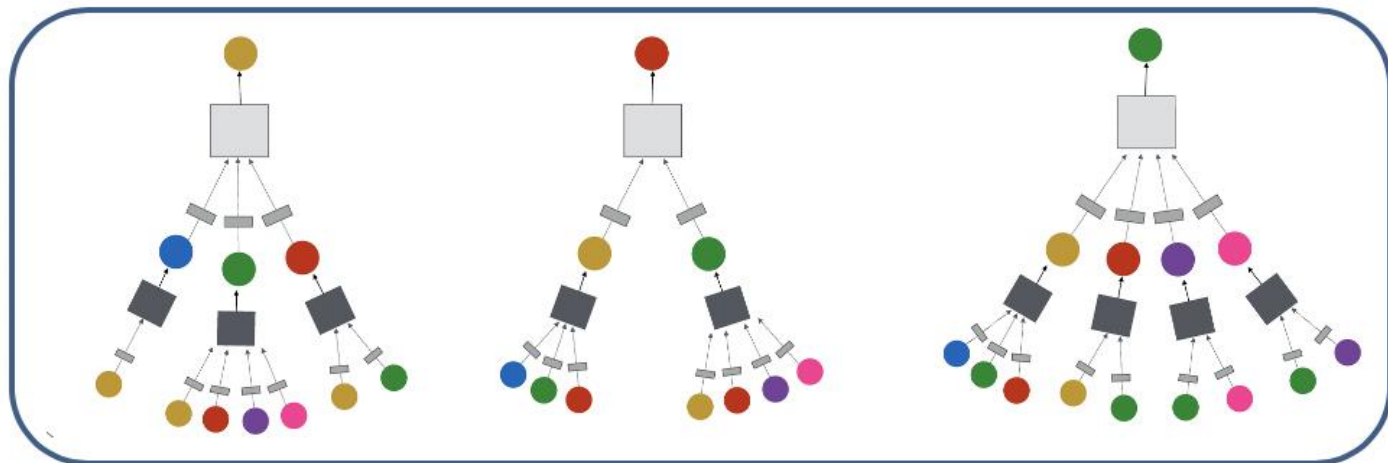
2) Define a loss function on the embeddings, $\mathcal{L}(z_u)$

Overview of Model Design

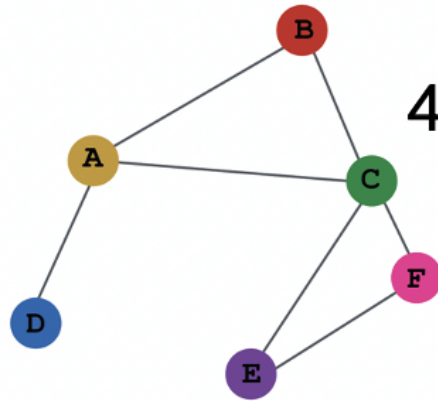


INPUT GRAPH

3) Train on a set of nodes, i.e., a batch of compute graphs



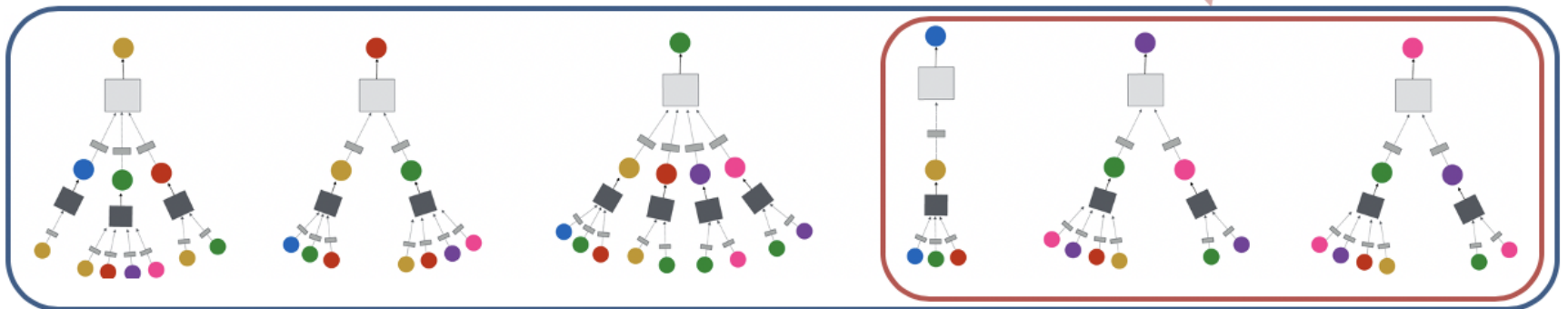
Overview of Model



INPUT GRAPH

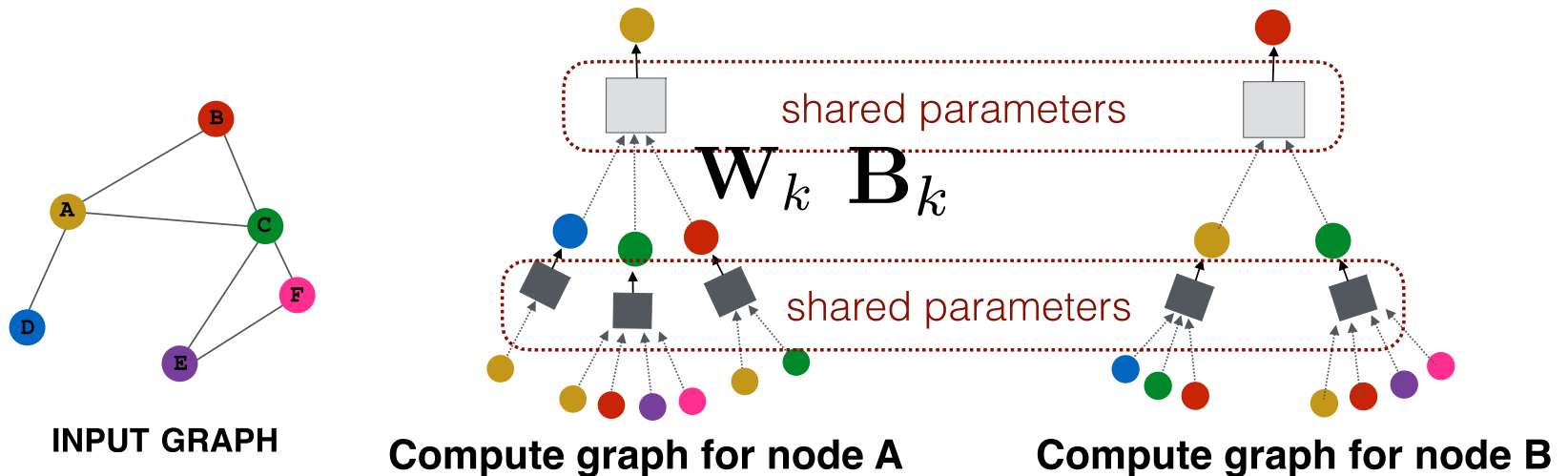
4) Generate embeddings for nodes as needed

Even for nodes we never trained on!!!!

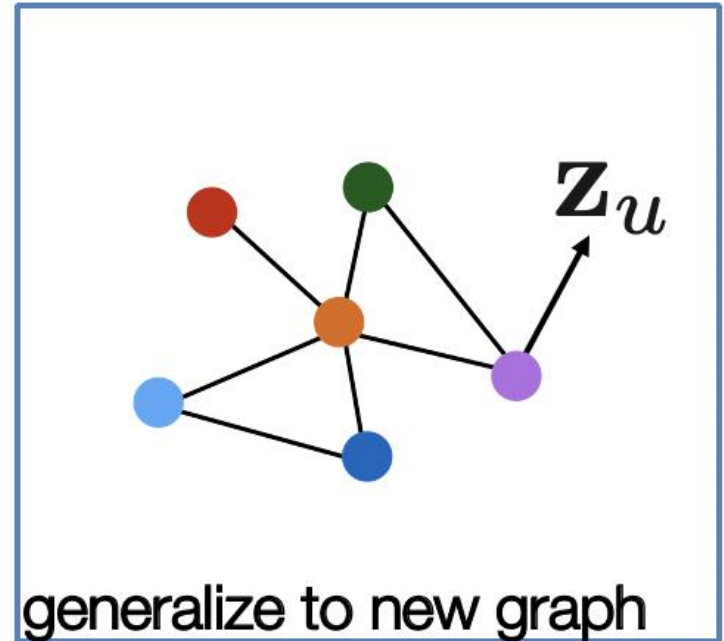
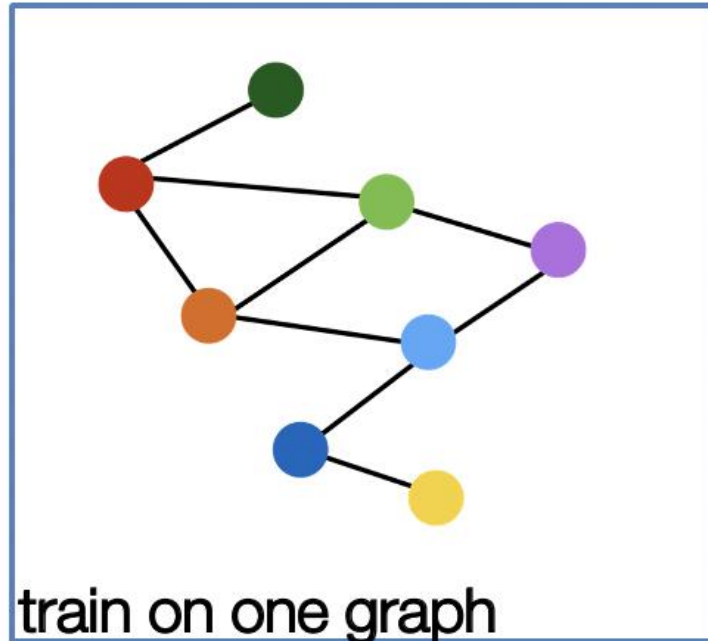


Inductive Capability

- The same aggregation parameters are shared for all nodes
- The number of model parameters is sublinear in $|V|$ and we can generalize to unseen nodes!



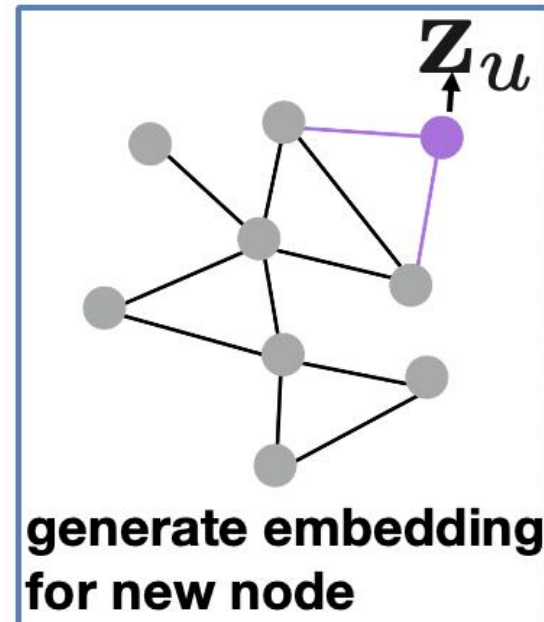
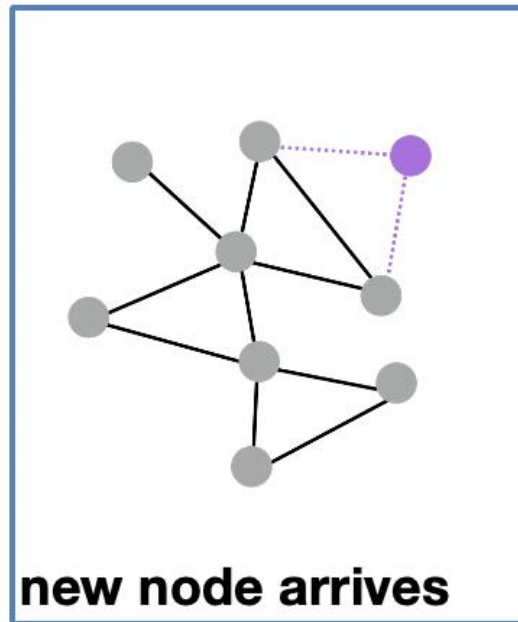
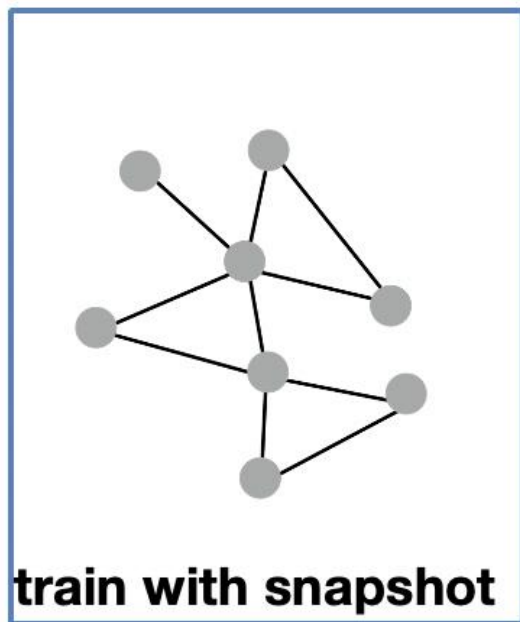
Inductive Capability on New Graphs



Inductive node embedding → generalize to entirely unseen graphs

e.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Inductive Capability on New Nodes



Many application settings constantly encounter previously unseen nodes.
e.g., Reddit, YouTube, GoogleScholar,

Need to generate new embeddings “on the fly”

Summary on GCN

- Key steps

- Design neural network by define aggregation function as

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right)$$

- Determine the loss function based on the tasks
 - Train the neural network with a set of nodes by optimizing the model parameters
 - Use trained parameters to generate embeddings for new nodes
- **Question:** Is there any other aggregation methods?

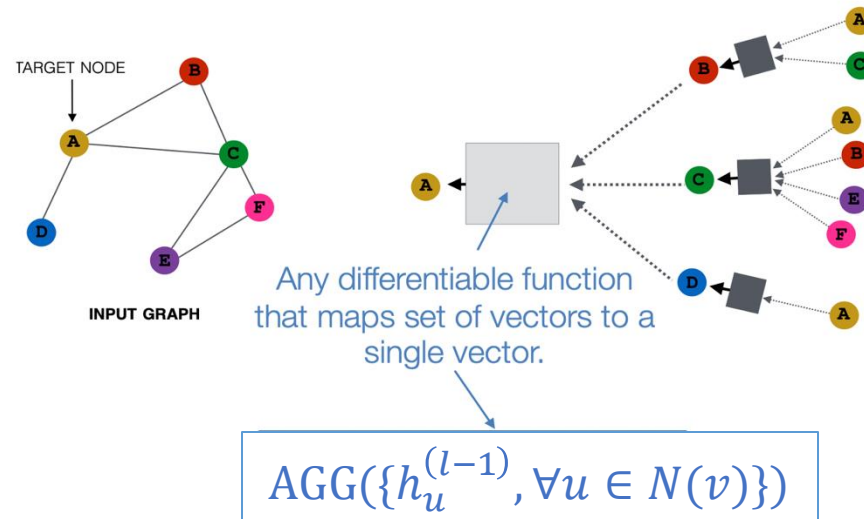
GraphSAGE

- Based on material from: Hamilton et al., 2017. [Inductive Representation Learning on Large Graphs](#). *NeurIPS*

GraphSAGE Idea

- Key idea
 - Generalized aggregation function
 - Concatenate aggregated embedding and self embedding

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT} \left(\mathbf{h}_v^{(l-1)}, \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right) \right) \right)$$



GraphSAGE Neighbor Aggregation

- **Mean:**

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$

- **Pool:** transform neighbor vectors and apply symmetric vector function

$$\text{AGG} = \text{Mean/Max}(\{\text{MLP}(h_u^{(l-1)}), \forall u \in N(v)\})$$

- **LSTM:** apply LSTM to randomly shuffled neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))])$$

GraphSAGE: L2 Normalization

- ℓ_2 Normalization (Optional)

$$\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\|\mathbf{h}_v^{(l)}\|_2} \quad \forall v \in V \text{ where } \|\mathbf{u}\|_2 = \sqrt{\sum_i u_i^2}$$

- Ensures all vectors have the same ℓ_2 norm
- In some cases, normalization results in performance improvement

Graph Attention Networks

- Based on material from: Veličković et al., 2018. Graph Attention Networks. ICLR

Graph Attention Network

- Neighborhood aggregation:

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

Attention weights

- In GCN/GraphSAGE
 - $\alpha_{vu} = 1/|N(v)|$ is the **weighting factor (importance)** of node u 's message to node v
 - $\Rightarrow \alpha_{vu}$ is defined explicitly based on the structural properties of the graph (node degree)
 - $\Rightarrow$ All neighbors are equally important

Graph Attention Network

- Attentive neighborhood aggregation:

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

Attention weights

- Not all node's neighbors are equally important
 - Attention is inspired by cognitive attention
 - The attention α_{uv} focuses on the important parts of the input data and fades out the rest
 - **Idea:** the NN should devote more computing power on that small but important part of the data
 - Which part of the data is more important depends on the context and is learned through training

Graph Attention Network

- **Key Questions**

- Can we do better than simple neighborhood aggregation?
- Can we let weighting factors α_{vu} to be implicitly defined?

- **Goal:** Specify **arbitrary importance** to different neighbors of each node in the graph
- **Idea:** Compute embedding h_v^k of each node in the graph following an **attention strategy**
 - Nodes attend over their neighborhoods' message
 - Implicitly specifying different weights to different nodes in a neighborhood

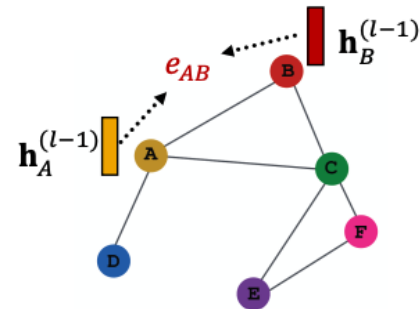
Attention Mechanism

- Let α_{vu} be computed as a byproduct of an **attention mechanism a**
 - Let a compute attention coefficients e_{vu} across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_v^{(l-1)})$$

- e_{vu} indicates the importance of node u 's message to node v

$$e_{AB} = a(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)})$$



Attention Mechanism

- Normalize e_{vu} into the final attention weight

α_{vu}

$$\sum_{u \in N(v)} \alpha_{vu} = 1$$

- Use the softmax function, so that

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

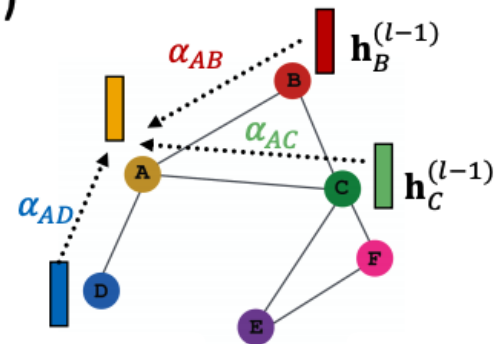
- Weighted sum based on the final attention weight

α_{vu}

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

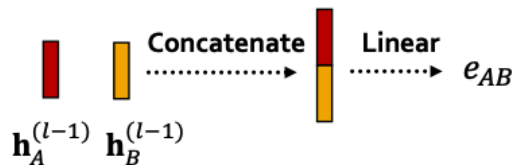
Weighted sum using α_{AB} , α_{AC} , α_{AD} :

$$\mathbf{h}_A^{(l)} = \sigma(\alpha_{AB} \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} + \alpha_{AC} \mathbf{W}^{(l)} \mathbf{h}_C^{(l-1)} + \alpha_{AD} \mathbf{W}^{(l)} \mathbf{h}_D^{(l-1)})$$



Attention Mechanism

- What is the form of attention mechanism a ?
- The approach is agnostic to the choice of a
 - E.g., use a simple single-layer neural network, a have trainable parameters (weights in the Linear layer)



$$\begin{aligned} e_{AB} &= a\left(\mathbf{W}^{(l)}\mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)}\right) \\ &= \text{Linear}\left(\text{Concat}\left(\mathbf{W}^{(l)}\mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)}\mathbf{h}_B^{(l-1)}\right)\right) \end{aligned}$$

- Parameters of a are trained jointly
Learn the parameters together with weight matrices (i.e., other parameter of the neural net $\mathbf{W}^{(l)}$) in an end-to-end fashion

Attention Mechanism

- **Multi-head attention:** Stabilize the learning process of attention mechanism
 - Create multiple attention scores (each replica with a different set of parameters):

$$\mathbf{h}_v^{(l)}[1] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^1 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[2] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^2 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[3] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^3 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

- Outputs are aggregated:
 - By concatenation or summation
 - $\mathbf{h}_v^{(l)} = \text{AGG}(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3])$

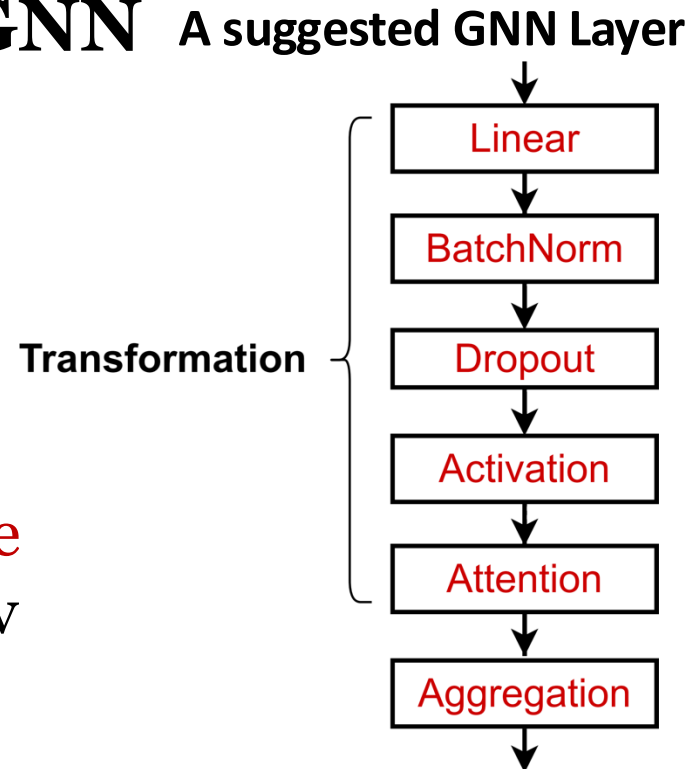
Benefits of Attention Mechanism

- **Key benefit:** Allows for (implicitly) specifying different important values α_{vu} to different neighbors
- **Computationally efficient:**
 - Computation of attentional coefficients can be parallelized across all edges of the graph
 - Aggregation may be parallelized across all nodes
- **Storage efficient:**
 - Sparse matrix operations do not require more than $O(V+E)$ entries to be stored
 - Fixed number of parameters, irrespective of graph size
- **Localized**
 - Only attends over local network neighborhoods
- **Inductive capability:**
 - It is a shared edge-wise mechanism
 - It does not depend on the global graph structure

GNN Layers in Practice

GNN Layer in Practice

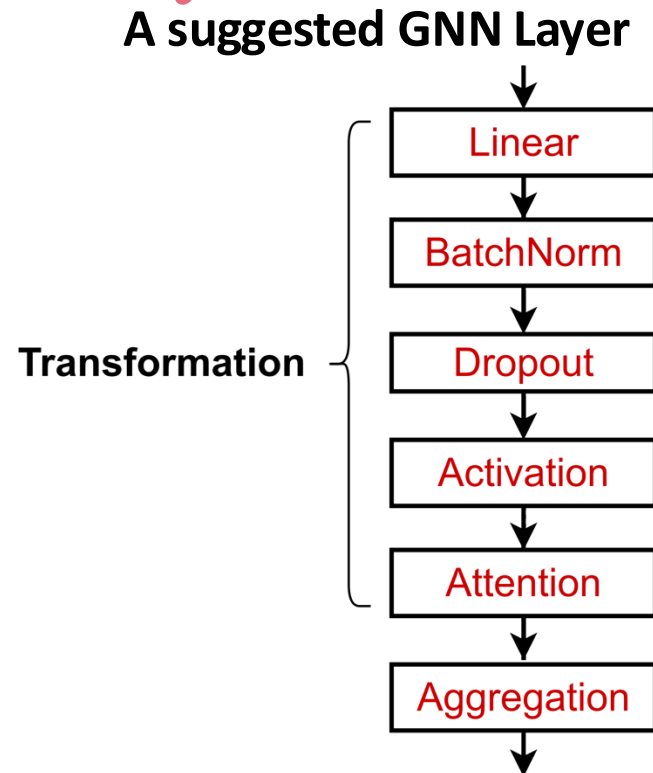
- **In practice, these classic GNN layers are a great starting point**
 - We can often get better performance by **considering a general GNN layer design**
 - Concretely, we can **include modern deep learning modules** that prove to be useful in many domains



GNN Layer in Practice

- **Many modern deep learning modules can be incorporated into a GNN layer**

- **Batch Normalization:**
 - Stabilize neural network training
- **Dropout:**
 - Prevent overfitting
- **Attention/Gating:**
 - Control the importance of a message
- **More:**
 - Any other useful deep learning modules



Batch Normalization

- Goal
 - Stabilize neural networks training
- Key idea (for a batch of inputs)
 - Re-center the node embeddings into zero mean
 - Re-scale the variance into unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times D}$
 N node embeddings

Trainable Parameters:

$\gamma, \beta \in \mathbb{R}^D$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$
Normalized node embeddings

Step 1:
Compute the mean and variance over N embeddings

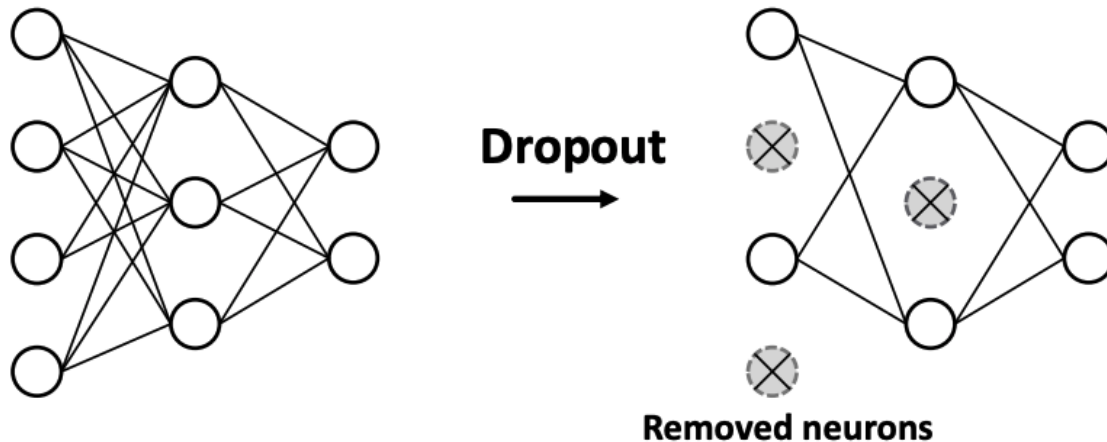
$$\begin{aligned}\mu_j &= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{i,j} \\ \sigma_j^2 &= \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{i,j} - \mu_j)^2\end{aligned}$$

Step 2:
Normalize the feature using computed mean and variance

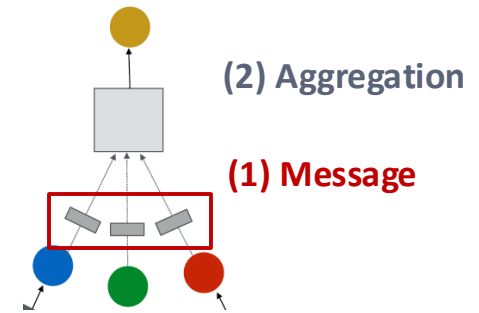
$$\begin{aligned}\hat{\mathbf{x}}_{i,j} &= \frac{\mathbf{x}_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \\ \mathbf{y}_{i,j} &= \gamma_j \hat{\mathbf{x}}_{i,j} + \beta_j\end{aligned}$$

Dropout

- Goal
 - Regularize a neural net to prevent overfitting
- Key idea
 - **During training:** with some probability p , randomly set neurons to zero (turn off)
 - **During testing:** Use all the neurons for computation

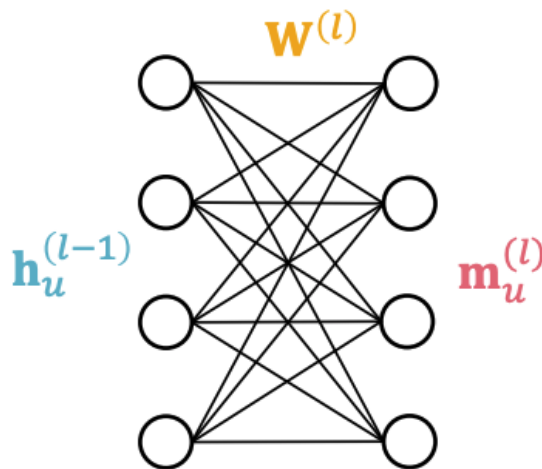


Dropout for GNNs



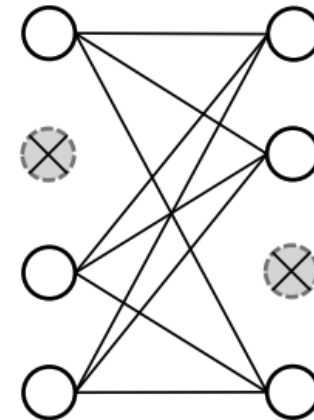
- In GNN, Dropout is applied to **the linear layer in the message function**
 - A simple message function with linear

$$\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$$



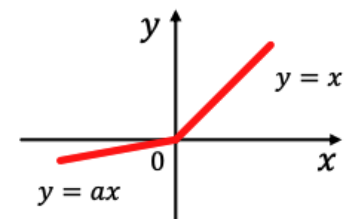
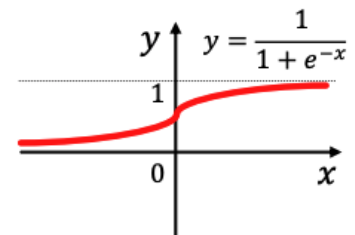
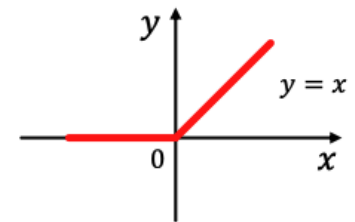
Visualization of a linear layer

Dropout
→



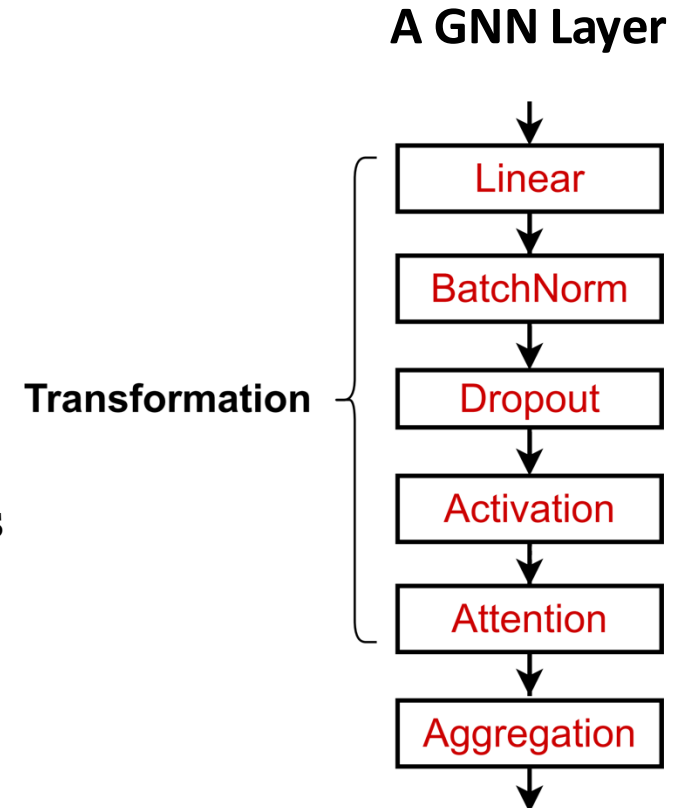
Activation (Non-linearity)

- Key idea:
 - Apply activation to i -th dimension of embedding x
- Activation functions
 - Rectified linear unit (ReLU)
 - $\text{ReLU}(x_i) = \max(x_i, 0)$
 - Most commonly used
 - Sigmoid
 - $\sigma(x_i) = \frac{1}{1+e^{-x_i}}$
 - Used when we need to restrict the range of the embeddings
 - Parametric ReLU
 - $\text{PReLU}(x_i) = \max(x_i, 0) + a_i \min(x_i, 0)$ (a_i is a trainable parameter)
 - Empirically performs better than ReLU



GNN Layer in Practice

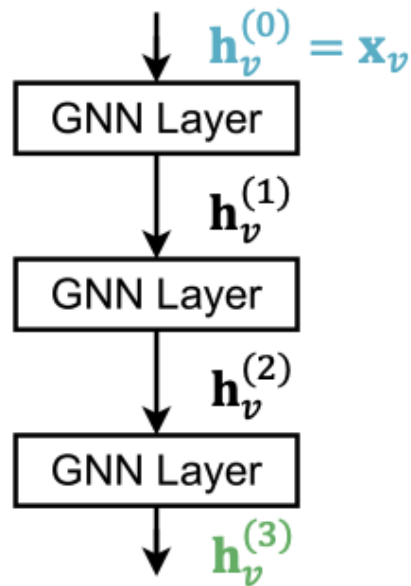
- Summary
 - Modern deep learning modules can be included into a GNN layer for better performance
 - Designing novel GNN layers is still an active research frontier
 - Suggested resources
 - You can explore diverse GNN designs or try out your own ideas



Stacking Layers of a GNN

Stacking GNN Layers

- Standard way: Stack GNN layers sequentially
 - **Input:** Initial node feature $\mathbf{x}_v$
 - **Output:** Node embeddings $\mathbf{h}_v^{(L)}$ after L GNN layers



The Over-smoothing Problem

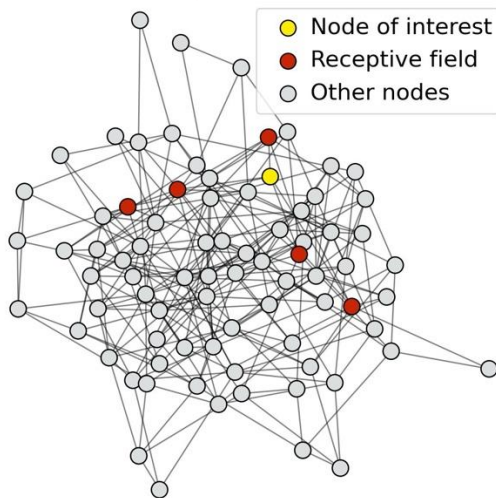
- The over-smoothing problem
 - All the node embeddings converge to the same value when stacking many GNN layers
 - Can greatly affect downstream tasks
- Question: Why over-smoothing happens?

Receptive Field of a GNN

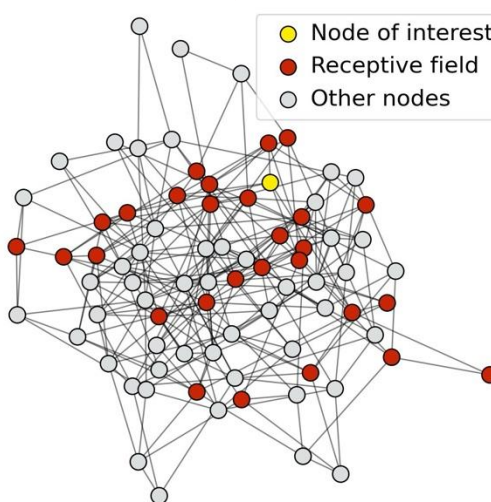
- Receptive field
 - The set of nodes that determine the embedding of a node of interest

In a K -layer GNN, each node has a receptive field of K -hop neighborhood

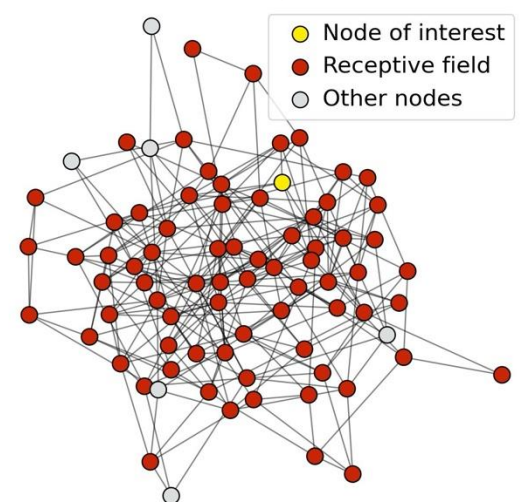
Receptive field for
1-layer GNN



Receptive field for
2-layer GNN



Receptive field for
3-layer GNN

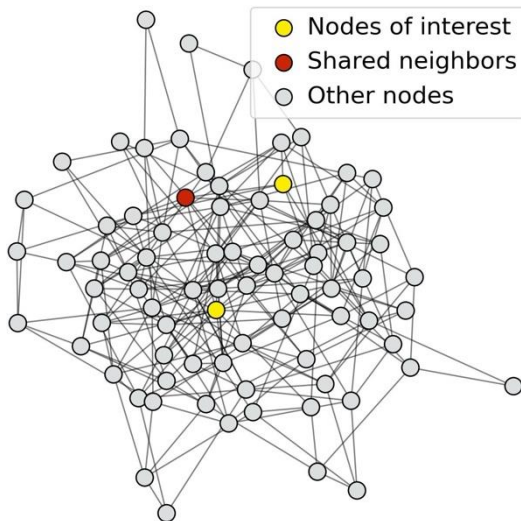


Receptive Field of a GNN

- Receptive field overlap between nodes
 - **The shared neighbors quickly grows** when we increase the number of hops (num of GNN layers)

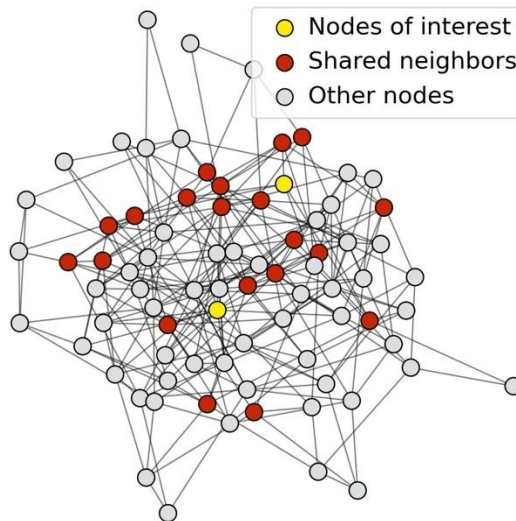
1-hop neighbor overlap

Only 1 node



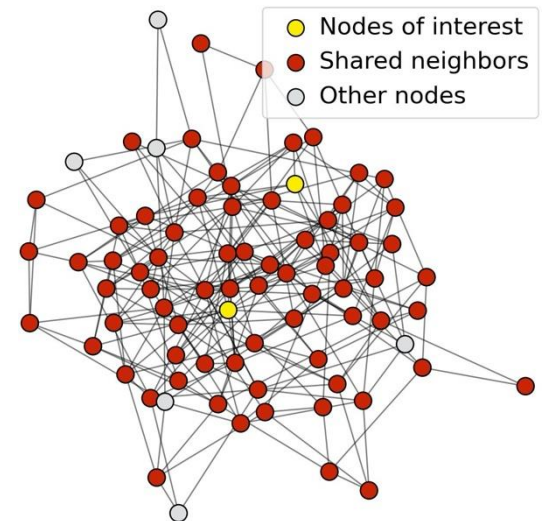
2-hop neighbor overlap

About 20 nodes



3-hop neighbor overlap

Almost all the nodes!

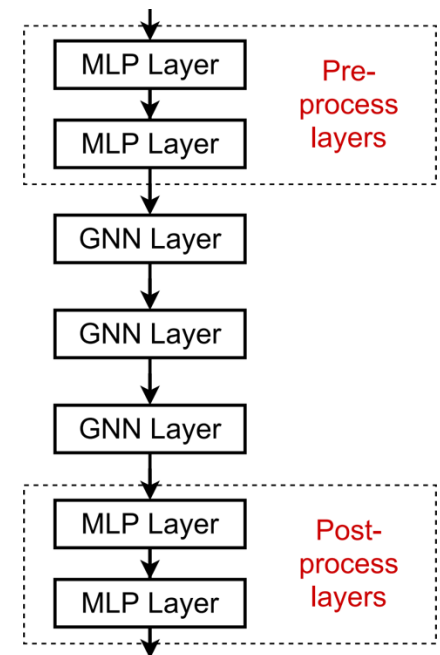
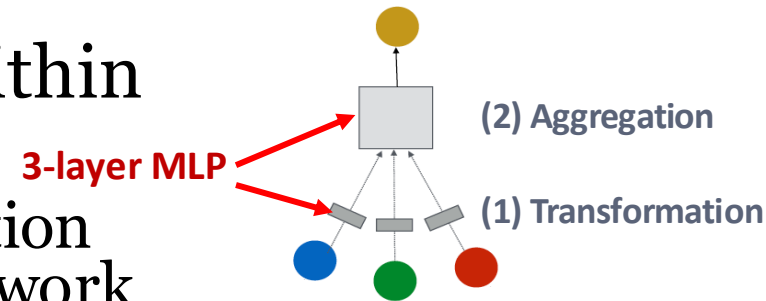


Receptive Field & Over-smoothing

- Over-smoothing
 - Stacking many GNN layers
 - → highly overlapped receptive fields between nodes
 - → node embeddings are high similar
 - → over-smoothing
- **Question:** How to tweak GNN to mitigate over-smoothing

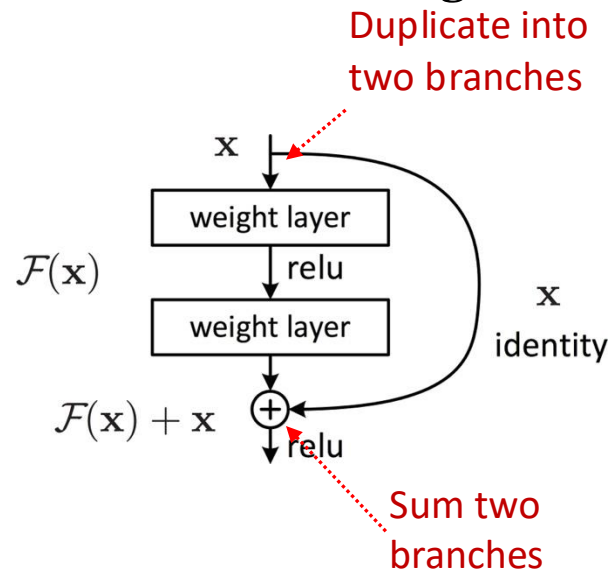
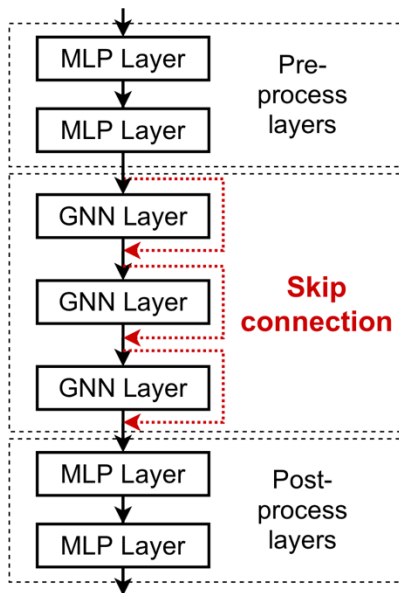
Solutions

- Increase expressive power within each GNN layers
 - Turn aggregation/transformation functions into deep neural network
- Add layers that do not pass messages
 - Pre-processing layers (**encoding node features, e.g. image/text node**)
 - Post-processing layers (**reasoning/transforming node embeddings, e.g. classification, knowledge graphs**)
- Add skip connections in GNNs



Skip Connections in GNNs

- Observations from over-smoothing
 - Node embeddings in earlier GNN layers can sometimes better differentiate nodes
- Key idea
 - **Add shortcuts in GNN** to increase the impact of earlier layers on the final node embeddings



Idea of skip connections:

Before adding shortcuts:

$$F \mathbf{x}$$

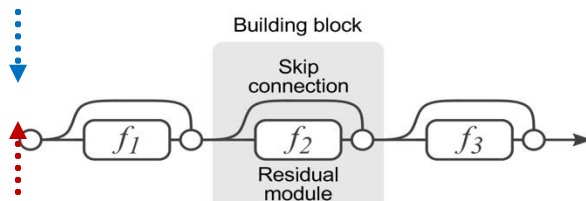
After adding shortcuts:

$$F \mathbf{x} + \mathbf{x}$$

Skip Connections in GNNs

- Why do skip connections work?
 - **Intuition:** skip connections create a mixture of models
 - N skip connections $\rightarrow 2^N$ possible paths
 - Each path could have up to N modules
 - We automatically get **a mixture of shallow GNNs and deep GNNs**

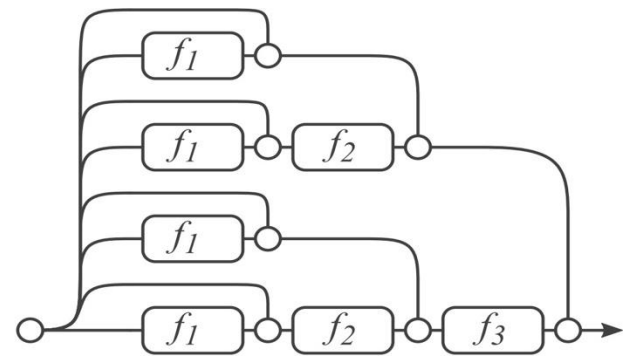
Path 2: skip this module



Path 1: include this module

(a) Conventional 3-block residual network

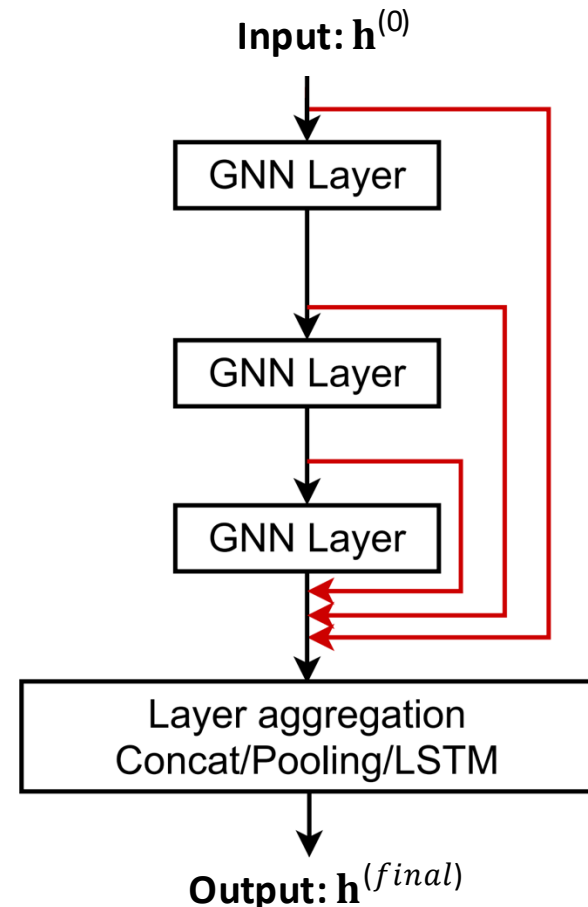
=



(b) Unraveled view of (a)

Other Options for Skip Connections

- Directly skip to the last layer
 - The final layer directly **aggregates from the all the node embeddings** in the previous layers





Questions?